Uma biblioteca, três responsabilidades

Visão geral • aproximadamente 5 minutos para esta parte da apresentação

Um único pool guarda os dados.
Uma lista encadeada registra os blocos ocupados.

03 mymemory_free

Retira uma alocação da lista e torna o intervalo reutilizável.

05 mymemory_stats

Mede ocupação e lacunas, incluindo as duas bordas do pool.

06 mymemory_cleanup

Encerra o gerenciador e devolve todos os recursos ao sistema.

Contrato: blocos de tamanho positivo, dentro do pool e sem sobreposição; nós alocados separadamente. A lista pode estar fora de ordem.

Como explicar: Minha parte cuida de liberar blocos, medir o estado da memória e encerrar a biblioteca. Init, alloc e display são a parte da dupla.

03 | Liberar uma alocação

mymemory_free • busca linear O(n) • memória extra O(1)

```
7    void mymemory_free(mymemory_t *memory, void *ptr)
8    {
9        if (memory == NULL || ptr == NULL)
10           return;
11
12       allocation_t **link = &memory->head;
13       while (*link != NULL) {
14           allocation_t *current = *link;
15           if (current->start == ptr) {
16               *link = current->next;
17               free(current);
18               return;
19           }
20           link = &current->next;
21       }
22   }
```

1. Validar e procurar

NULL é ignorado. A comparação aceita somente o endereço exato de início do bloco.

2. Reconectar a lista

link é um ponteiro para ponteiro: aponta para head ou para o next do nó anterior. *link = current->next pula o nó encontrado.

3. Liberar o controle

free(current) devolve o nó de controle. O pool permanece alocado. Se o endereço não estiver na lista, nada muda.

Exemplo: A → B → C passa a A → C
Na cabeca, a mesma operacão atualiza memory->head.

Como explicar: Não chamo free(ptr): ele aponta para dentro do pool. Removo o registro para que o alocador possa reconhecer esse espaço como livre.

05 | Contar blocos e bytes

mymemory_stats • primeira parte: totais e posição do primeiro bloco

```
33   size_t count = 0;
34   size_t used = 0;
35   size_t first = memory->total_size;
36   size_t largest = 0;
37   size_t fragments = 0;
38   const unsigned char *base = memory->pool;
39
40   for (allocation_t *node = memory->head; node != NULL; node = node->next) {
41       size_t start = (size_t)((const unsigned char *)node->start - base);
42       count++;
43       used += node->size;
44       if (start < first)
45           first = start;
```

Relação central: memória livre = total_size - used.
A função pressupõe os registros válidos definidos no contrato.

count e used

count conta os nós ativos. used soma os tamanhos registrados, sem incluir o custo dos próprios nós de controle.

Posição em bytes

unsigned char * permite calcular a distância em bytes entre start e o início do pool. size_t representa tamanhos; %zu é seu formato de impressão.

first

Começa no tamanho total e recebe o menor início encontrado. Se não houver alocações, continua igual ao tamanho do pool.

Como explicar: A lista informa os blocos ocupados. Eu somo seus tamanhos e calculo onde cada um começa dentro do pool.

05 | Medir cada lacuna livre

mymemory_stats • segunda parte: achar o próximo bloco pela posição no pool

```
48   size_t next_start = memory->total_size;
49   for (allocation_t *other = memory->head; other != NULL; other = other->next) {
50       size_t offset = (size_t)((const unsigned char *)other->start - base);
51       if (offset > start && offset < next_start)
52           next_start = offset;
53   }
54
55   size_t gap = next_start - (start + node->size);
56   if (gap > 0) {
57       fragments++;
58       if (gap > largest)
59           largest = gap;
60   }
```

Custo: O(n²), pois percorre a lista para cada bloco.
Aceita lista desordenada sem alterá-la. Com ordem garantida, seria possível usar O(n).

Próximo no pool

A busca interna encontra o menor início maior que start. Se não existir outro bloco depois, next_start continua sendo o fim do pool.

Tamanho da lacuna

start + node->size é o fim do bloco atual. Subtrair esse fim do próximo início fornece gap.

Fragmentos e maior bloco

Só lacunas positivas entram na contagem. largest guarda a maior. O trecho seguinte do código também conta a lacuna inicial, usando first.

Como explicar: O next da lista não precisa ser o próximo endereço na memória. Por isso procuro o vizinho pela posição dentro do pool.

05 | Fragmentação em um exemplo

Pool de 1.000 bytes • os intervalos usam início inclusivo e fim exclusivo

Lista usada no teste: 600 → 100 → 300. A ordem da lista é diferente da ordem no desenho.



Resultado esperado

3 alocações
400 bytes ocupados
600 bytes livres

Distribuição do espaço livre

4 fragmentos: 100, 100, 200 e 200 bytes.
Maior bloco contíguo: 200 bytes.

Como explicar: Há 600 bytes livres ao todo, mas nenhum intervalo tem mais de 200. Um pedido de 250 bytes não cabe sem mover as alocações existentes.

06 | Encerrar o gerenciador

mymemory_cleanup • percurso O(n) • memória extra O(1)

```
78  void mymemory_cleanup(mymemory_t *memory)
79  {
80      if (memory == NULL)
81          return;
82
83      allocation_t *node = memory->head;
84      while (node != NULL) {
85          allocation_t *next = node->next;
86          free(node);
87          node = next;
88      }
89      free(memory->pool);
90      free(memory);
91  }
```

1. Guardar antes de liberar

Salvo node->next em next antes de free(node). Depois de free, acessar os campos desse nó seria uso de memória liberada.

2. Percorrer toda a lista

node = next avança usando o ponteiro salvo. Até os nós de blocos ainda ativos são liberados no encerramento.

3. Liberar pool e gerenciador

Primeiro free(memory->pool), depois free(memory). No chamador, atribua memory = NULL; os ponteiros antigos deixam de ser válidos.

Como explicar: Free remove um único registro. Cleanup encerra a biblioteca inteira, inclusive os blocos ainda registrados, o pool e a estrutura principal.

O que foi verificado

Testes isolados das funções 3, 5 e 6 • não substituem os testes do alocador completo

Cenário	Alocações	Usados	Livres	Maior livre	Fragmentos
Vazio	0	0 B	1.000 B	1.000 B	1
Fragmentado	3	400 B	600 B	200 B	4
Cheio	2	1.000 B	0 B	0 B	0
50 blocos	50	458 B	542 B	19 B	50

Remoção: cabeça, meio, cauda, último nó, NULL, endereço externo, ponteiro interno e liberação repetida sem reutilização do endereço.

Resultado: compilação C11 com -Wall -Wextra -Wpedantic -Werror, sem avisos; asserts e conferência automática das cinco estatísticas passaram.

Como explicar: Os testes montam registros manualmente. Ainda falta integrar init, alloc e display e testar as estratégias de alocação e o reaproveitamento real das lacunas.

Roteiro de fala e perguntas frequentes

Use como apoio de estudo • explique cada operação com suas próprias palavras

Roteiro sugerido • 5 minutos

30 s: pool e lista.
1 min: free e ponteiro para ponteiro.
1 min 40 s: stats e exemplo de fragmentação.
50 s: cleanup e ordem de liberação.
1 min: testes e integração.

O que ainda precisa ser integrado

Itens 1, 2 e 4 da dupla. No main fornecido, usar mymemory_alloc e mymemory_cleanup, conforme o cabeçalho; verificar NULL antes de acessar os dados.

Por que não liberar start?

start aponta para uma região do pool; não é um malloc independente. Somente o início original do pool é liberado no encerramento.

Espaço livre é igual a espaço contíguo?

Não. A soma das lacunas pode ser grande, mas cada pedido precisa caber em uma região contínua.

E um ponteiro antigo após reutilização?

A API identifica blocos pelo endereço. Se ele for reutilizado, não distingue um ponteiro antigo de um atual. O chamador não deve reutilizar ponteiros liberados.

Fontes: enunciado T2, seção 2.2; mymemory.h; implementação e testes do repositório catharinar/T2-ProgSoftBas. Os números ao lado do código são as linhas de mymemory.c.

Uso de IA: funções 3, 5 e 6, testes e material explicativo preparados com auxílio do Codex. Revisão e compreensão pelos integrantes são necessárias para a apresentação.